

Maestría en Ciencias de Datos e Innovación Empresarial

Aprendizaje Automático

(Deep Learning)

Prof. Dr. Marcelo Raponi

Unidad I

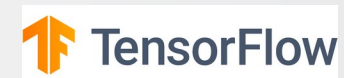
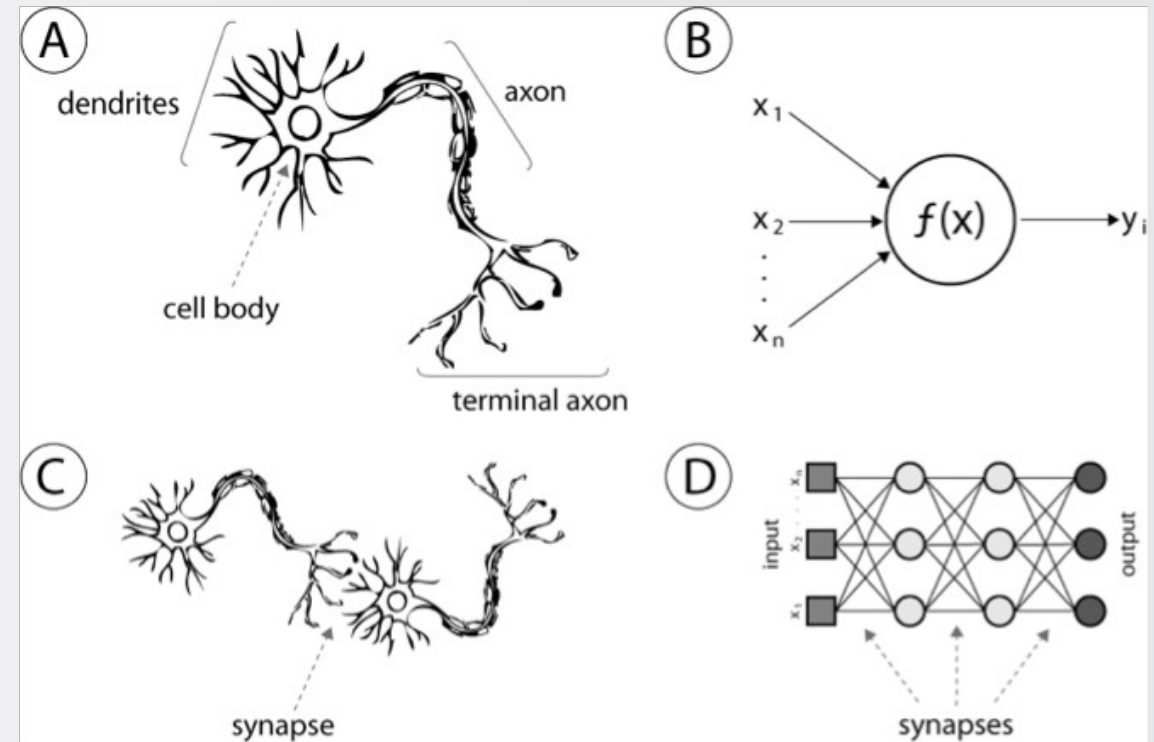
Temario Unidad 1

- Repaso: regresión y clasificación.
- Función de pérdida.
- Función de activación.
- Perceptrón.
- Aprendizaje por gradiente descendiente.
- Red neuronal artificial.
- Parámetros e hiperparámetros.
- Frameworks: TensorFlow-Keras, Pytorch.

Material complementario:

* Utilización de IDEs: VSCode, Jupyter Notebook, Google Colab

* Procesamiento de Imágenes empleando Python y OpenCV



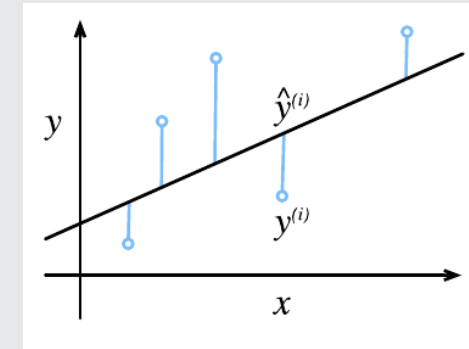
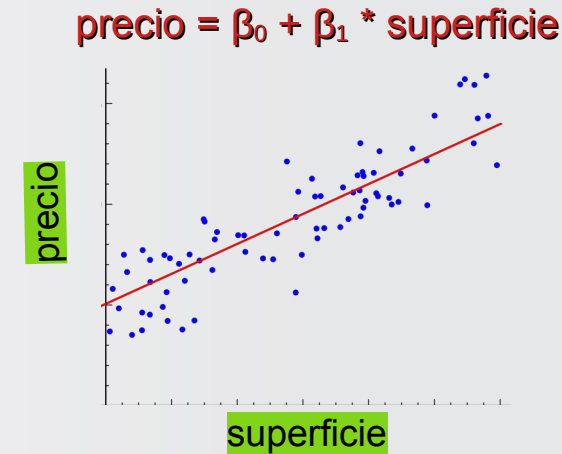
Regresión

A partir de una o más variables predictoras y una variable de respuesta, el algoritmo busca determinar la relación entre dichas variables, que permita predecir un valor cuantitativo.

* **Regresión Lineal simple:** $\hat{y} = \beta_0 + \beta_1 x + \varepsilon$

- x es la variable independiente (predictora o explicativa),
- y es la variable dependiente (respuesta continua)
- $\hat{y}_i$ es el valor *predicho* por el modelo

El algoritmo de regresión determina los parámetros del modelo (β_0 y β_1) que minimizan el error cuadrático medio (**MSE**, Mean Squared Error).



$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

* **Regresión Lineal múltiple:** idem al caso simple, pero ahora existe más de una variable independiente.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_m x_m + \varepsilon \quad \Rightarrow \quad \boldsymbol{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

Ec. Normales

$x_1, x_2, \dots, x_m$: features

$\hat{y}_i$: valor de salida predicho

ε : término de error

β_0 : intercept, bias, offset

β_1 : coef de x_1 (primer feature)

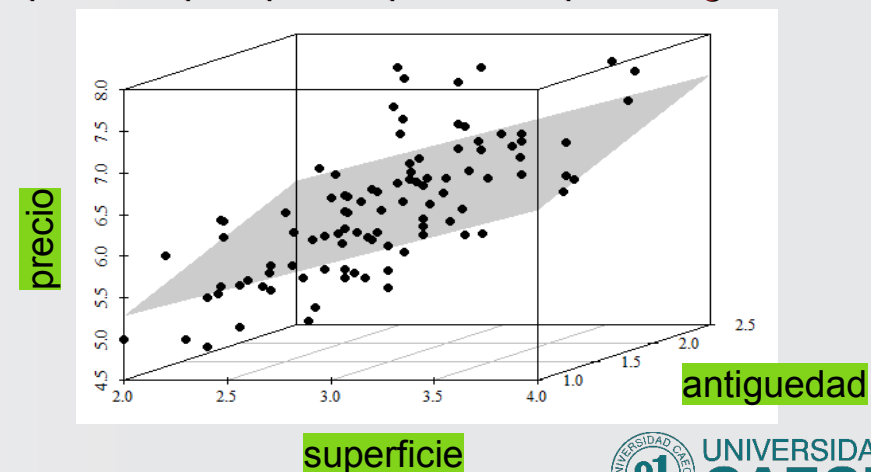
β_m : coef de x_m (m - ésimo feature)

$$X = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \dots & x_{1,m} \\ 1 & x_{2,1} & x_{2,2} & \dots & x_{2,m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n,1} & x_{n,2} & \dots & x_{n,m} \end{bmatrix}$$

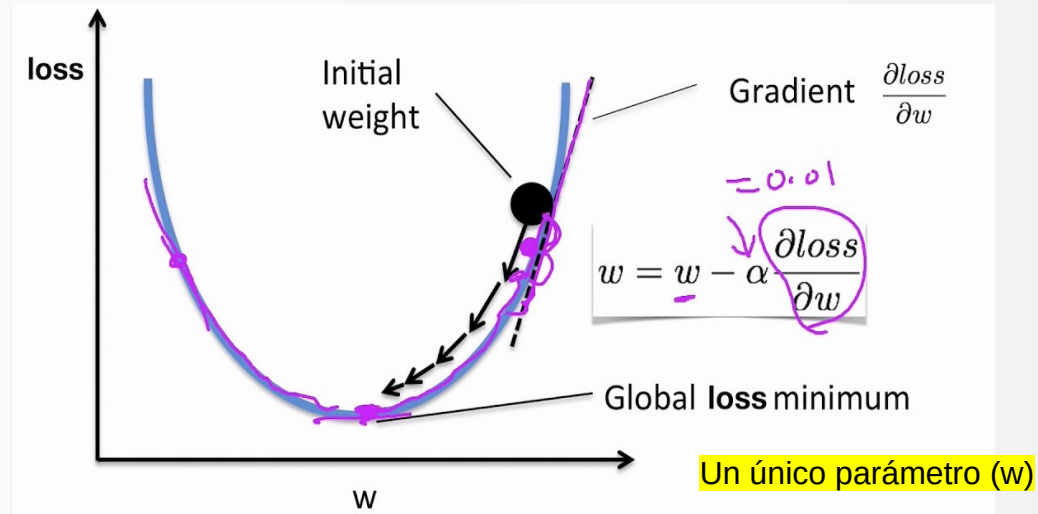
Matriz de diseño

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

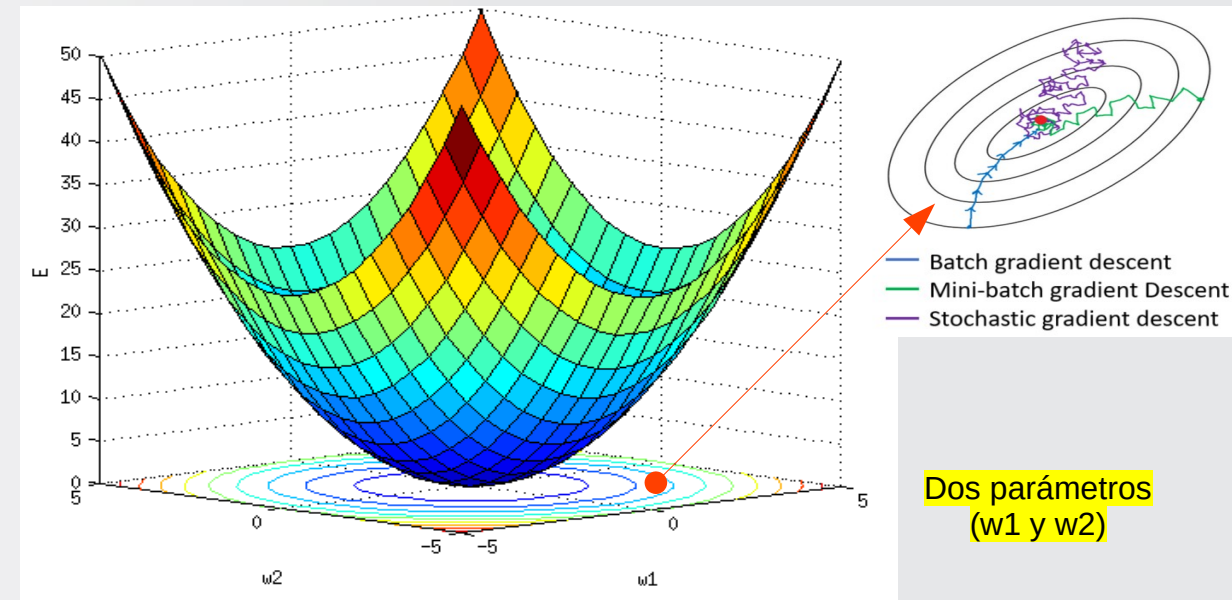
precio = $\beta_0 + \beta_1 * \text{superficie} + \beta_2 * \text{antigüedad}$



Función de pérdida (loss)



En cada iteración damos un paso en **dirección opuesta a la del gradiente** (nos movemos descendiendo en la *loss* para acercarnos al mínimo).



Clasificación

El objetivo es predecir las **etiquetas** de nuevas instancias (nuevos datos) basado en observaciones pasadas, a diferencia de los algoritmos de regresión que predicen una cantidad.

* **Clasificación binaria:** el algoritmo aprende a distinguir entre dos clases.

* **Clasificación multiclase:** el algoritmo aprende a distinguir entre más de dos clases.

Ej. reconocimiento de caracteres escritos a mano (**dataset MNIST**). El conjunto de datos de entrenamiento (training set) consta de múltiples ejemplos de números (0-9) escritos a mano.

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x})$$

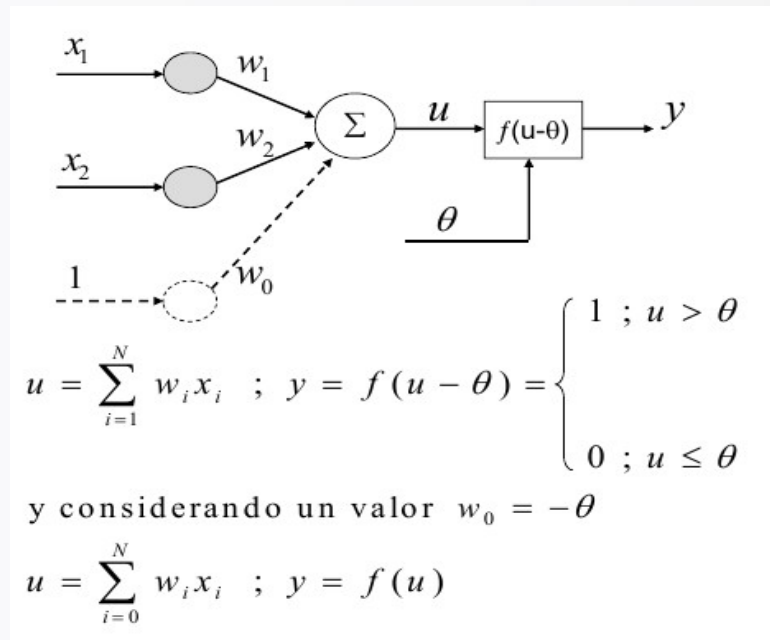
Regression

$$y(\mathbf{x}, \mathbf{w}) = f \left(w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) \right)$$

Classification

Perceptrón

- **1957, Frank Rosenblatt:** Presentación del perceptrón, un algoritmo de aprendizaje automático utilizado para resolver problemas de clasificación binaria.
- El perceptrón simple sólo puede resolver problemas de clasificación linealmente separables, es decir, cuando las dos categorías pueden separarse por una línea recta en un espacio de características.
- La neurona recibe estímulos (entradas) que son ponderados (pesados) y sumados generando una señal de salida neta, que luego pasa a través de una función de activación.
- **Función activación:** escalón $\rightarrow \{0, 1\}$, signo $\rightarrow \{-1, 1\}$ Introduce no linealidad en la operación de la neurona.
- **1969, Minsky-Papert:** En el libro Perceptrons demuestran que el perceptrón es incapaz de resolver problemas no lineales como el XOR. Comienza la era oscura de las redes neuronales.
- **1980s.** Regreso de las redes neuronales gracias al algoritmo de retropropagación del error.



w_k = Eficacia de las sinapsis (peso), donde

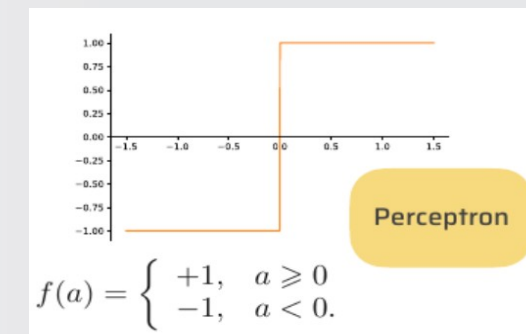
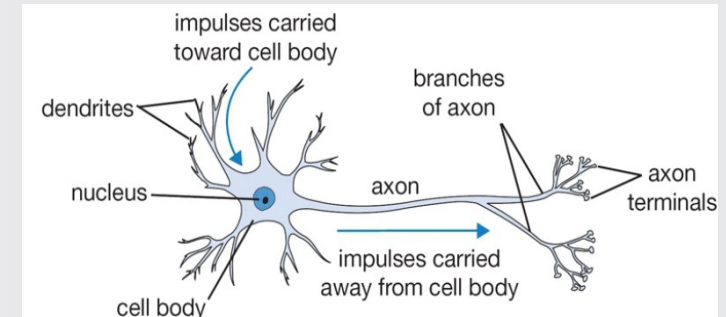
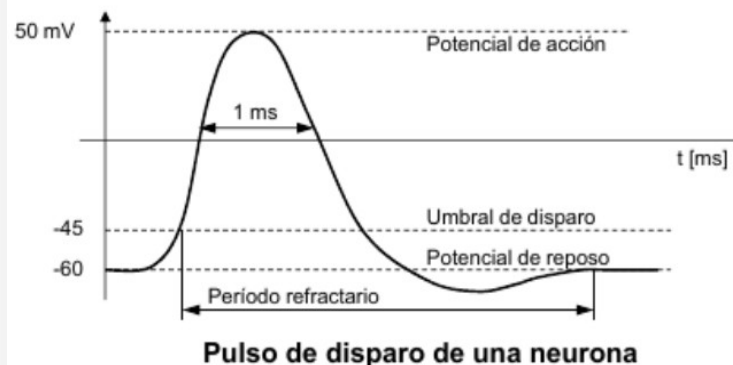
$w_k > 0$ sinapsis excitadora

$w_k < 0$ sinapsis inhibidora

$u = \sum w_k \cdot x_k$ (Potencial Pos Sináptico)

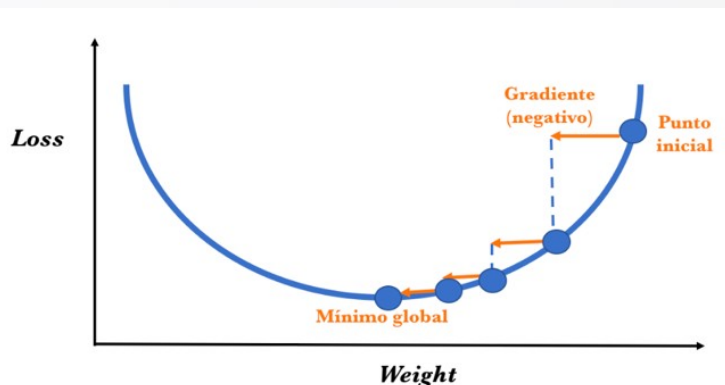
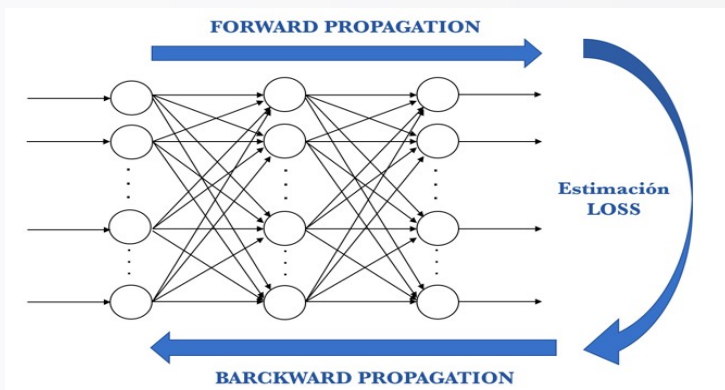
θ = Umbral de activación

$s = f(v) = f(u - \theta)$ (salida de la neurona)



Redes Neuronales Artificiales: perceptrón multicapa (MLP)

- Si bien están inspiradas en el cerebro humano y en cómo sus neuronas interactúan entre sí, las Redes Neuronales Artificiales o RNA (en inglés, Artificial Neural Networks - ANN) no están destinadas a ser modelos realistas del cerebro.
- Cada neurona de una red MLP incluye una función de activación no lineal que es diferenciable (**sigmoide**).
- Posee una o más **capas ocultas** y nodos de salida. Las conexiones siempre están dirigidas **hacia adelante** (**forward**), no hay realimentación. Las conexiones están afectadas por un factor o “peso” sináptico.
- Las neuronas tienen un **umbral** que puede considerarse una entrada más, de intensidad unitaria y un cierto peso (w_0).
- Todas las unidades de una capa están conectadas a todas las unidades de la capa siguiente (**fully-connected**).



Backpropagation:

Técnica efectiva de entrenamiento propuesta por **David Rumelhart, Geoffrey Hinton y Ronald Williams** en **1986**. *Propaga el error de adelante hacia atrás para ajustar los pesos.*

Función de pérdida (loss), función de costo o función objetivo:

*Medida que cuantifica la distancia entre el valor real (target) y el valor predicho por la red neuronal, mientras está en el proceso de entrenamiento. Ej: **MSE** (regresión), **cross-entropy** (clasificación)*

Optimizador:

*Algoritmo encargado de minimizar la función de pérdida. Ej: **SGD**, **Adam**, **Adagrad**, etc.*

Gradiente descendente

Recordemos!! la **derivada** de una función $f(x)$ en un punto, es la **pendiente** de la recta tangente a dicho punto (medimos la inclinación en cada tramo de nuestra función).

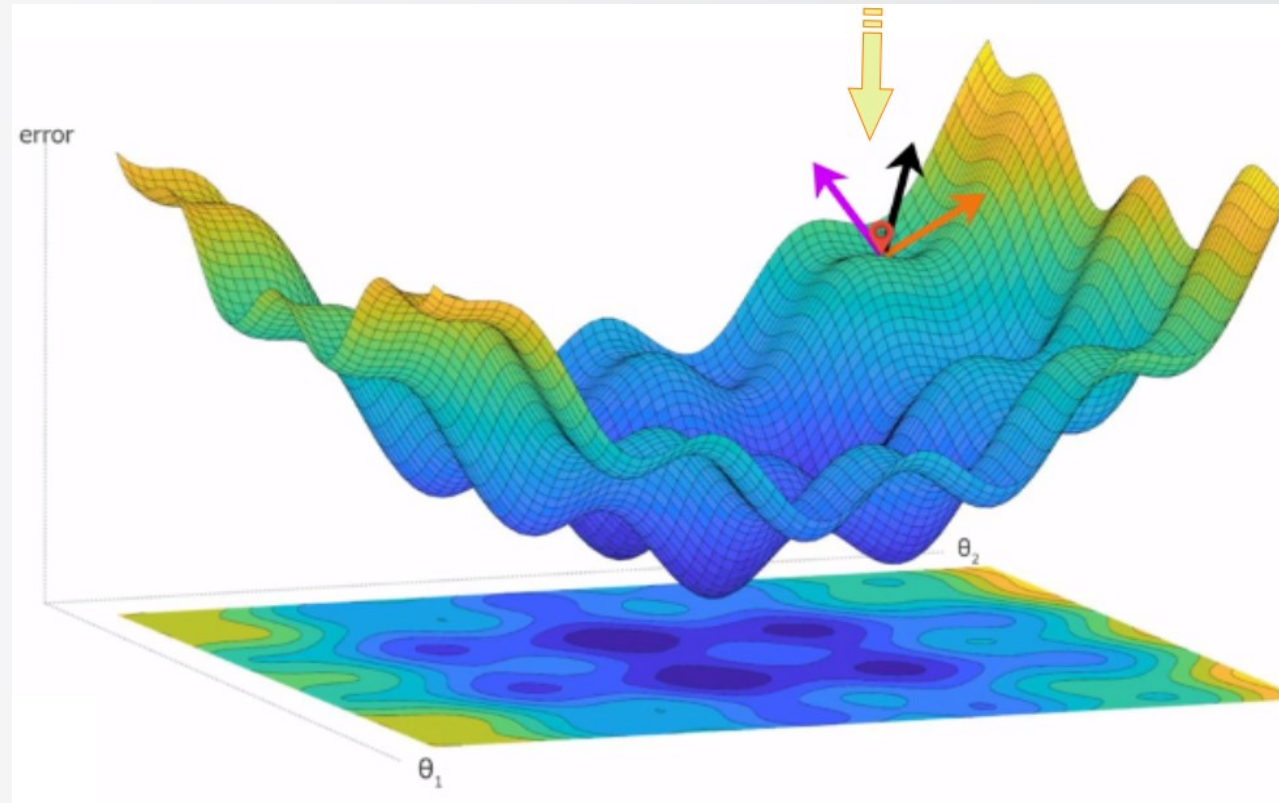
Búscamos el mínimo de la superficie de error en el **hiperespacio de pesos** mediante métodos de **descenso por gradiente**. Podemos caer en un mínimo local.

Como nuestra función de pérdida es 3D, calculamos las derivadas parciales respecto a c/parámetro, obteniendo el gradiente:

$$\nabla_{loss} = \begin{bmatrix} \frac{\partial loss}{\partial \theta_1} \\ \frac{\partial loss}{\partial \theta_2} \end{bmatrix}$$

$$\theta_{k+1} = \theta_k - \alpha \cdot \nabla_{loss}$$

α (**learning rate**): nos dice cuanto avanzamos en c/iteración



Aleatoriamente nos encontramos en un punto sobre la superficie.

Evaluamos la pendiente en dicha posición (calculamos el gradiente de la función en dicho punto).

¿Cómo se entrena una red?

Proceso iterativo que consiste en:

- Inicializar los parámetros de la red (pesos w_{ij} y sesgos b_j).
- Tomar un conjunto de datos y pasarlos por la red para obtener una predicción.
- Comparar predicciones vs etiquetas (labels) y calcular la pérdida (loss).
- Retropropagar la pérdida a todos los parámetros que conforman el modelo.
- Actualizar los parámetros de la red.
- Repetir hasta que consideremos que tenemos un buen modelo.

Matriz de confusión

Tabla que contabiliza las predicciones en comparación con los valores reales. Muy útil para mostrar de forma explícita “cuando una clase es confundida con otra”.

		Actual Labels	
		1	0
Predicted Labels	1	True Positive	False Positive
	0	False Negative	True Negative

Matriz de confusión para un clasificador binario

TP: # de positivos clasificados correctamente.

TN: # de negativos clasificados correctamente.

FN: # de positivos clasificados incorrectamente como negativos.

FP: # de negativos clasificados incorrectamente como positivos.

		Clase real	
		1 (gato)	0 (perro)
Clase predicha	1 (gato)	TP= 130	FN= 30
	0 (perro)	FN= 20	TN= 100

Accuracy (exactitud): predicciones correctas (tanto TP como TN) sobre el total de predicciones. Es un valor para toda la red. No siempre es apropiada para evaluar la performance del modelo.

Precision (precisión): poder predictivo positivo. De los gatos detectados cuantos realmente eran gatos. Cada clase tiene un valor.

Recall (sensitivity): tasa de identificación correcta de elementos positivos. Cuantos gatos detectó correctamente sobre el total de gatos.

Specificity (especificidad): tasa de identificación correcta de elementos negativos. Cuantos perros detectó correctamente sobre el total de perros.

F1 score: media armónica de precisión y sensibilidad.

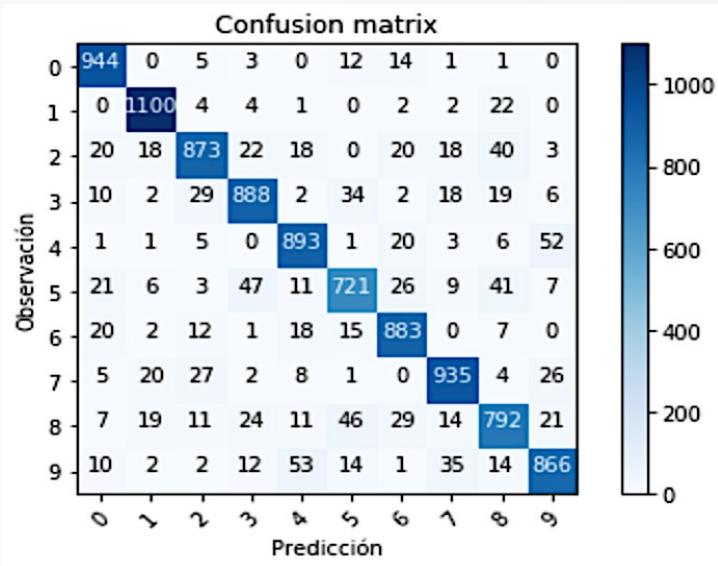
$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$
$$\text{F1 score} = 2 \times \frac{(\text{Prec} \times \text{Rec})}{(\text{Prec} + \text{Rec})}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{False +ve rate} = \frac{\text{FP}}{\text{TN} + \text{FP}}$$
$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}}$$
$$\text{Recall, Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

True +ve rate

Matriz de confusión para un clasificador multiclase



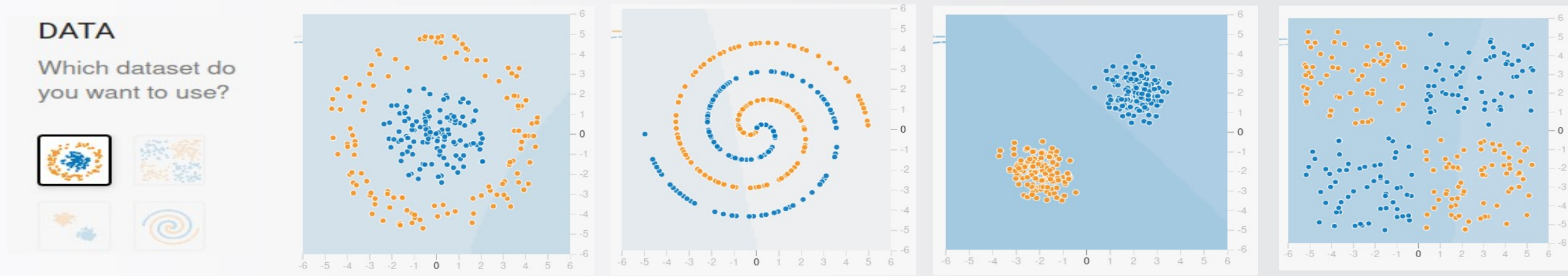
- Los elementos de la diagonal representan el número de muestras en que la etiqueta que predice el modelo coincide con el valor real.
- Los otros valores nos indican los casos en que el modelo ha clasificado incorrectamente.
- Cuanto más altos son los valores de la diagonal mejor será la predicción.

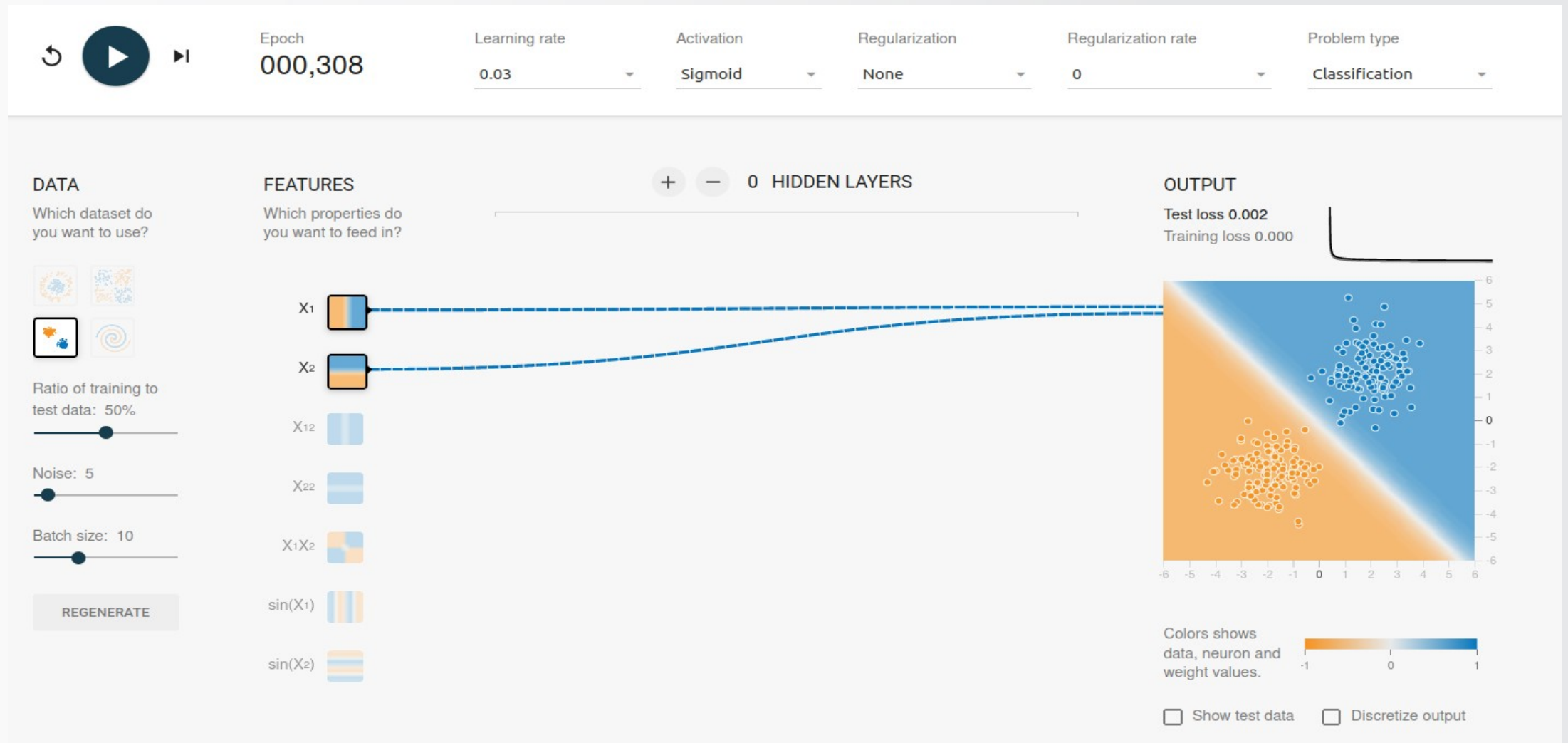
Clasificación binaria usando red neuronal

TensorFlow Playground (<http://playground.tensorflow.org>) es una aplicación web de visualización interactiva escrita en JavaScript que nos permite simular redes neuronales simples y ver los resultados en tiempo real.

Podemos experimentar con diferentes hiperparámetros y ver su comportamiento.

Primero debemos elegir el conjunto de datos indicado en el apartado “DATA”, luego botón “**Play**”



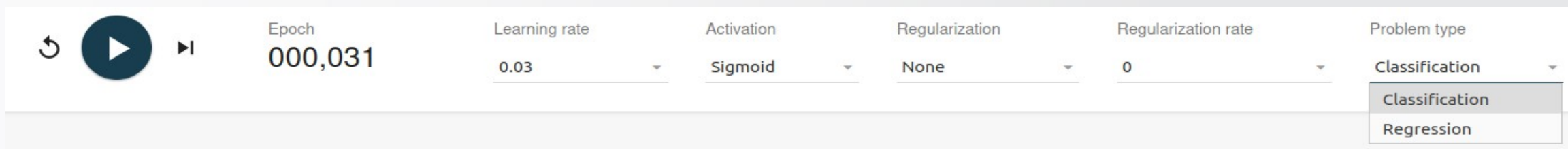


La línea entre la zona de color azul y naranja empieza a moverse lentamente. Puede pulsar el botón de “Reset” y volver a probarlo varias veces para ver como la línea se mueve con diferentes valores iniciales a medida que el **contador epoch** va aumentando.

La aplicación trata de encontrar los mejores valores de los parámetros que permitan clasificar correctamente estos puntos. Si pone el cursor encima de los **arcos**, aparecerán los valores asignados a cada parámetro (también permite editarlo y probarlo para ver su efecto)

Menú superior: puedo seleccionar diferentes **hiperparámetros** (Epoch, Learning rate, Activation, Regularization, Regularization rate) y distintas **tareas** (Problem type).

Problem type: la plataforma permite especificar dos tipos de problema: **Regresión** y **Clasificación**.



The screenshot shows a configuration bar for a machine learning task. It includes a reset button, a play button, and a pause button. The settings are: Epoch 000,031, Learning rate 0.03, Activation Sigmoid, Regularization None, Regularization rate 0, and Problem type Classification. A dropdown menu for Problem type is open, showing 'Classification' and 'Regression' options. An orange arrow points to the 'Classification' option.

Podemos elegir entre cuatro tipos de datos para clasificación y dos para regresión. Los puntos azules y naranjas forman el dataset.

* Los puntos **naranjas** tienen el valor **-1**

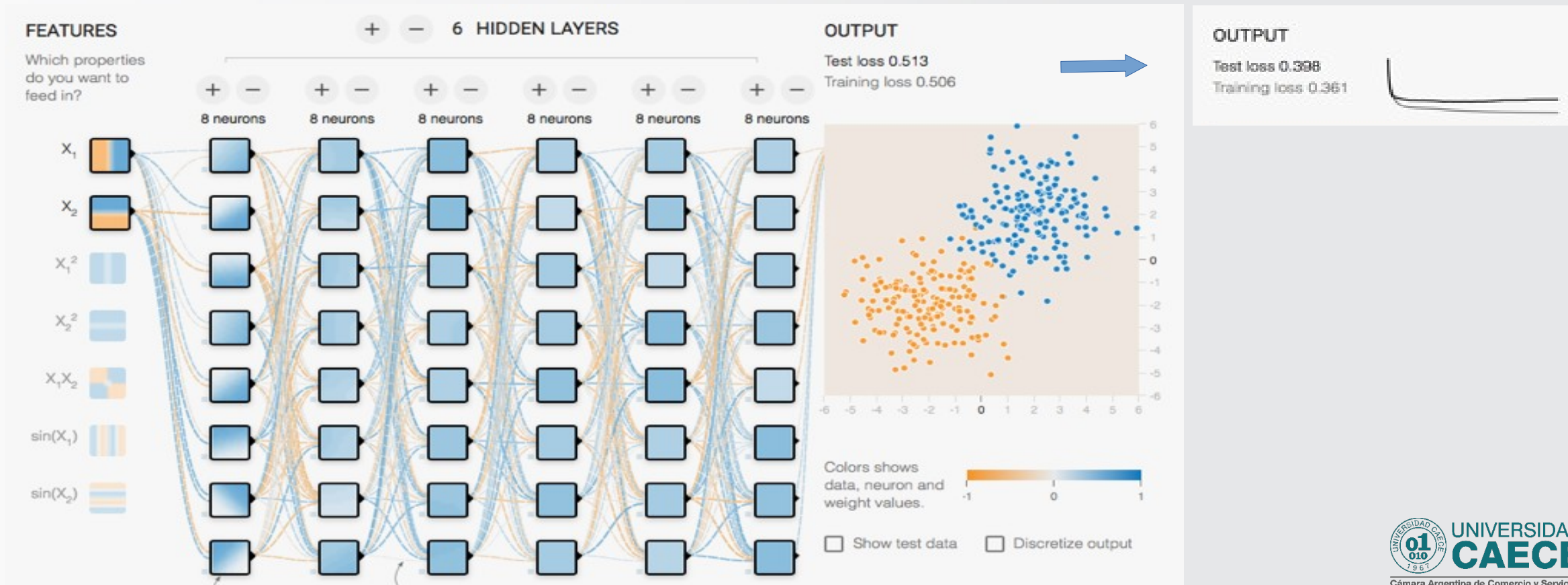
* Los puntos **azules** tienen el valor **+1**



Hay diferentes parámetros que podemos modificar para **tunear** nuestros datos de entrada.

- **Ratio of training to test data**: le indicamos el porcentaje de datos que se asignarán al conjunto de entrenamiento y al de testing (al modificar vemos como cambian los puntos en “OUTPUT”).
- **Noise**: nivel de ruido de los datos. El patrón de datos se vuelve más irregular a medida que aumenta el ruido. Ruido = 0, los datos se distinguen claramente en sus regiones, ruido > 50 los puntos azules y los puntos anaranjados se mezclan, es muy difícil clasificarlos.
- **Batch size**: determinamos la cantidad de datos que se usará para cada *lote* de entrenamiento.

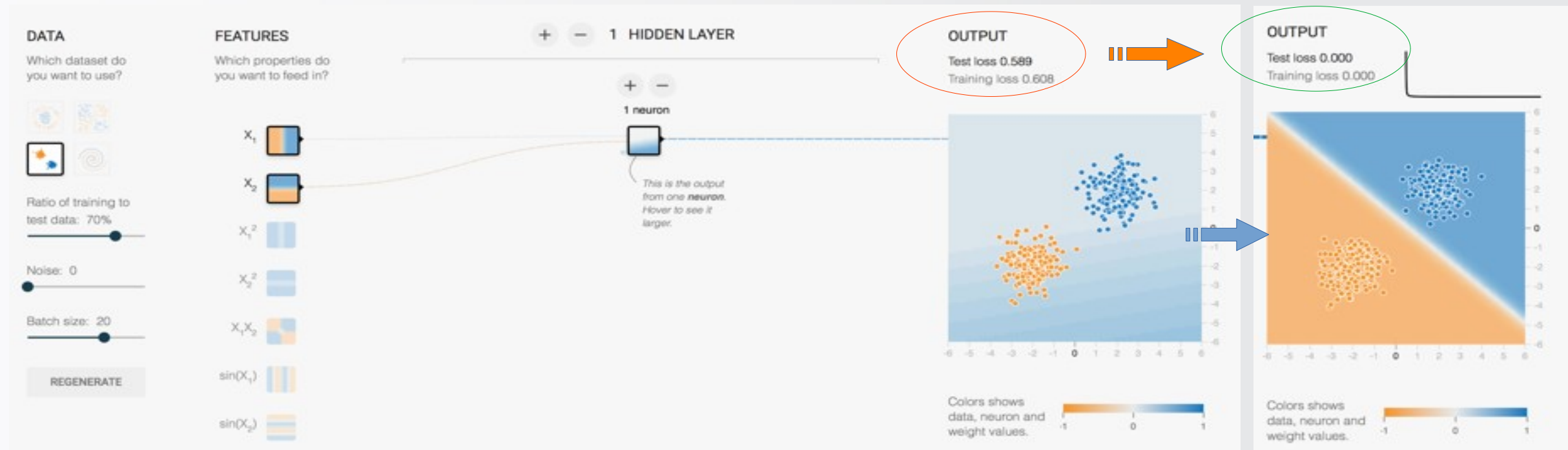
- Podemos seleccionar diferentes características (**features**). Usemos “X1” (eje horizontal) y “X2” (eje vertical).
- La **topología** de la red se puede definir agregando capas. Podemos tener **hasta seis capas ocultas** (HIDDEN LAYERS, clic en el signo “+”).
- Podemos tener **hasta ocho neuronas por capa oculta** (clic en el signo “+” de la capa correspondiente)
- Al entrenar la red neuronal queremos minimizar la “**Training loss**” (*curva gris*) y luego ver que con los datos de testing también se minimiza la “**Test loss**” (*curva negra*). Se observarán cambios de ambas métricas a medida que procesa cada **epoch**.



Clasificación con una sola neurona

Clasificación que separa en dos grupos los datos. Usemos un “**learning rate**” de 0.03 y una “**función de activación**” de tipo **ReLU**. El problema es de clasificación, fijamos el “**ratio of training-to-test**” a 50% y “**noise**” en **cero** para facilitar la visualización.

El “**batch size**” lo podemos dejar en 20, y usaremos “X1” y “X2” para la entrada. Empezaremos usando una capa oculta de una sola neurona (perceptron).

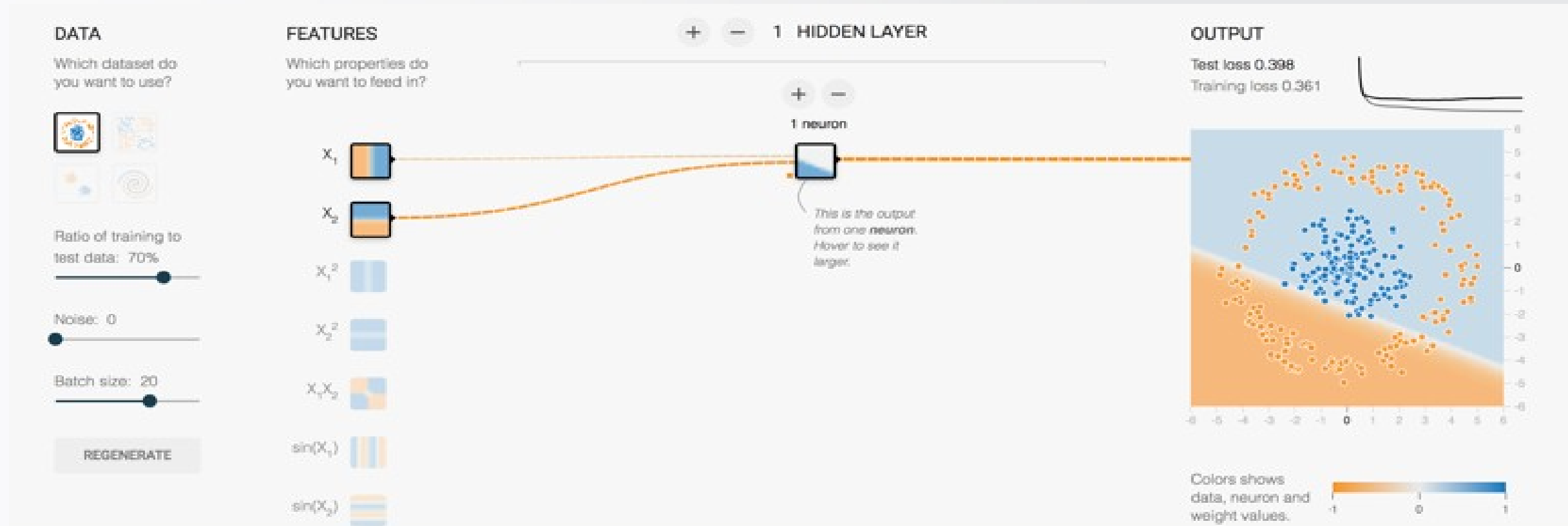


Vemos que los valores iniciales de la “*Test loss*” y “*Training loss*” son altos.

Después de pulsar el botón “**Play**” ambas curvas convergen a unos ratios muy bajos y se mantienen.

Además, en este caso, ambas curvas (negra y gris) se superponen perfectamente.

Escojamos otro conjunto de datos de partida.



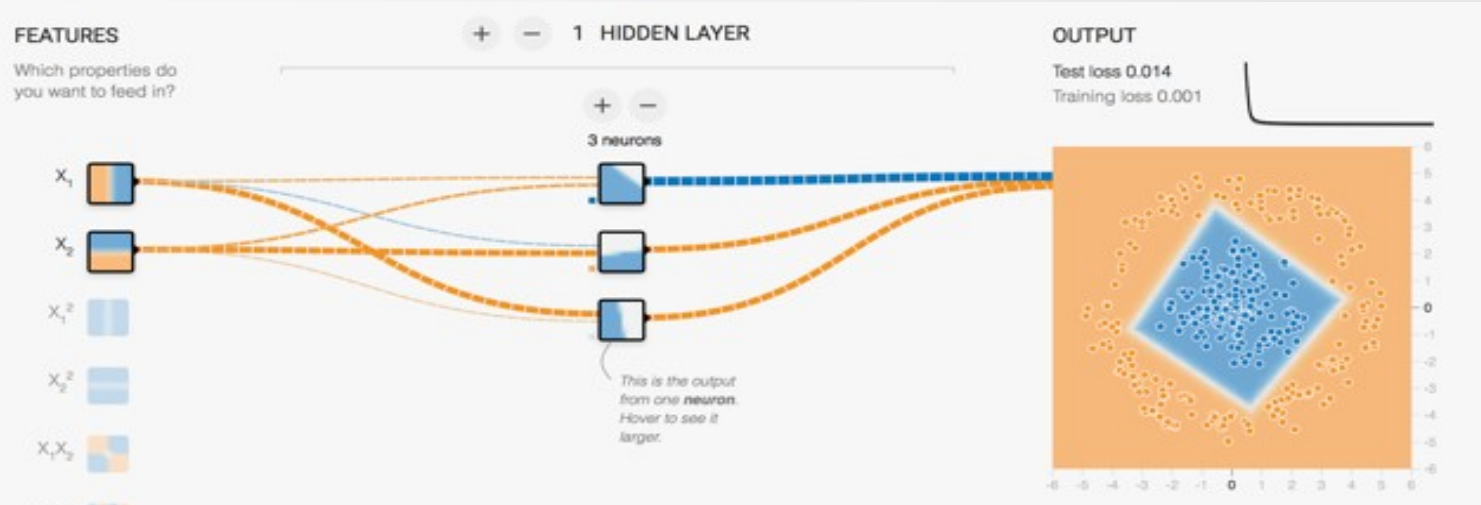
Queremos ahora separar los dos conjuntos de datos. En este problema los puntos de color naranja están en un círculo exterior y los azules en su interior (como planeta y órbita).

Estos datos no se pueden separar con una línea como antes. Trabajando con una sólo neurona la tarea de clasificación fallará.

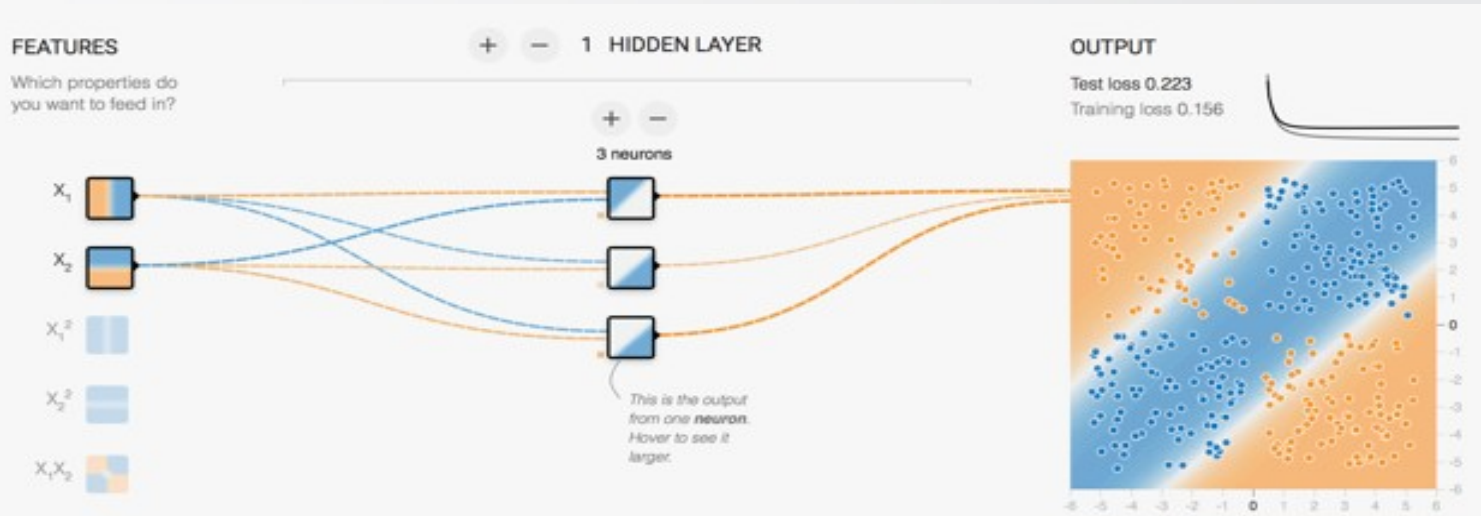
Probaremos entonces con múltiples neuronas en la capa oculta.

Clasificación con más de una neurona

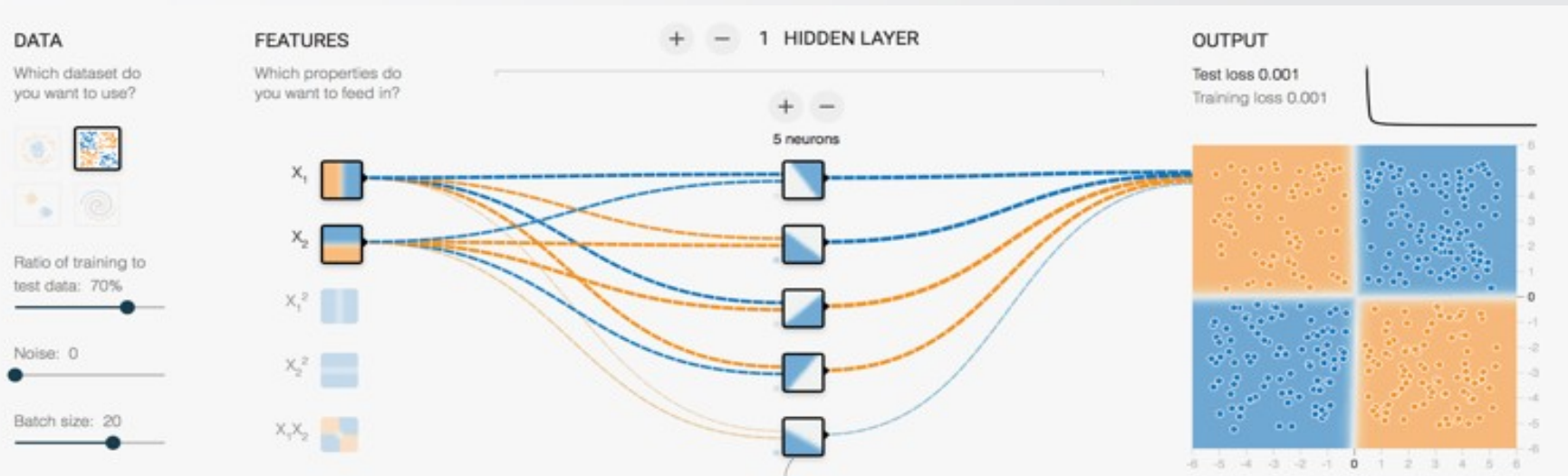
Si usamos dos neuronas, veremos que aún no clasifica bien. Probemos con tres neuronas en la capa oculta. Vemos que esta red consigue separar los datos y obtener una *loss* tanto de *training* como de *test* mucho mejor.



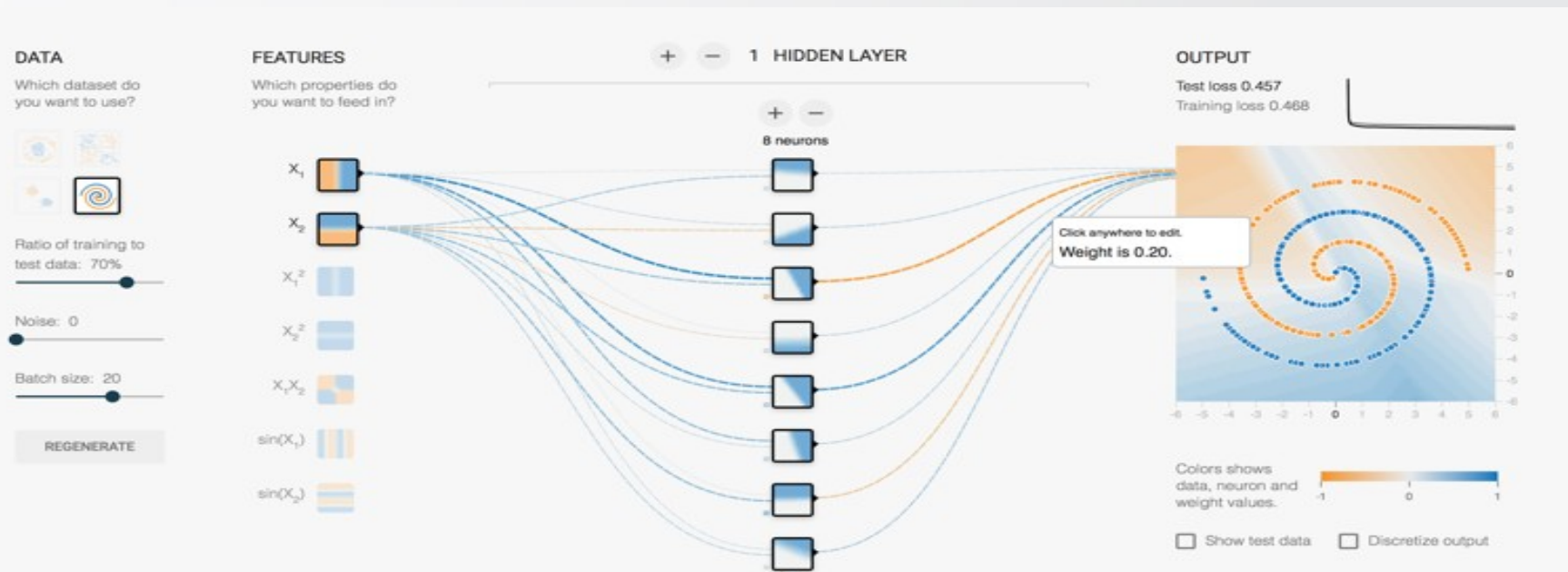
Cambiamos el dataset, ahora los datos están divididos en cuatro zonas diferentes. Este problema no se puede solucionar con la anterior red. Podría suceder que con 3 neuronas funcione dado que la inicialización es aleatoria, pero si repiten varias veces verán que no se consigue en general.



Si tenemos 5 neuronas, puede ver cómo esta red neuronal consigue una buena clasificación:

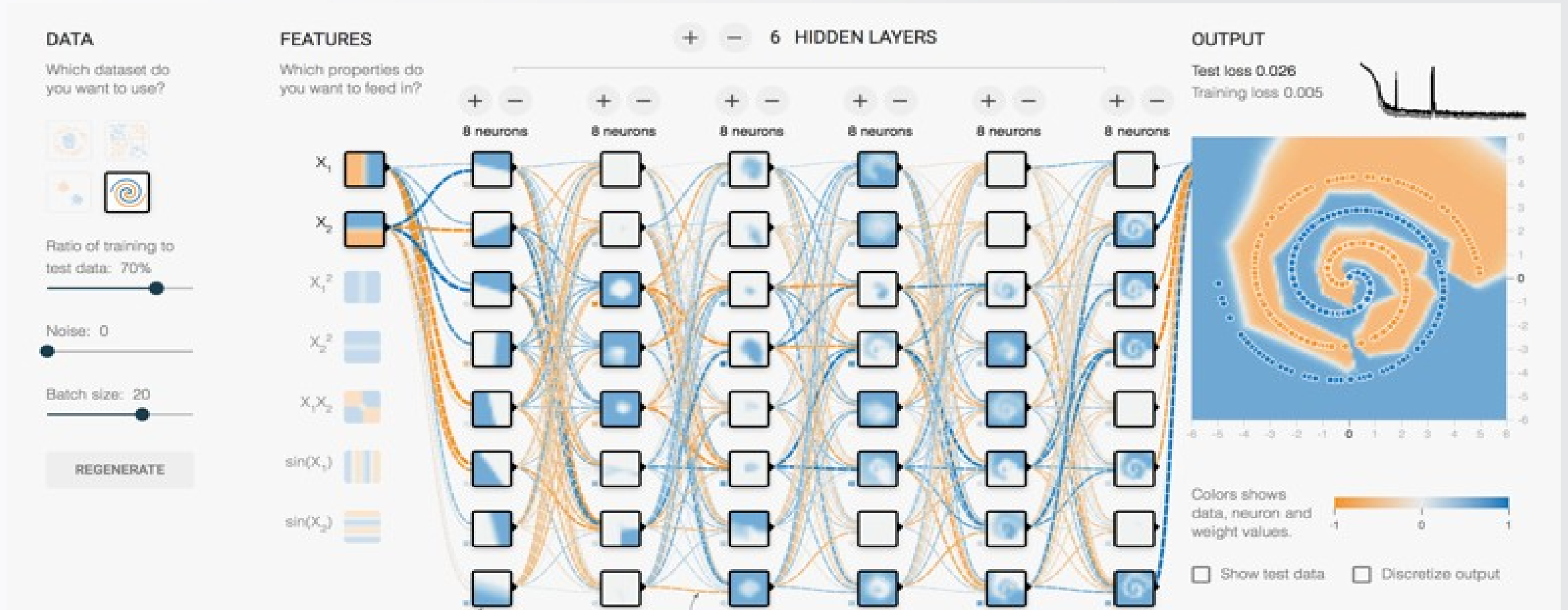


Seleccionemos ahora el dataset con un patrón más complejo. El remolino de puntos naranjas y azules es un problema desafiante. Con la red anterior, ni usando 8 neuronas (el máximo que nos deja la herramienta), conseguimos un buen resultado de clasificación

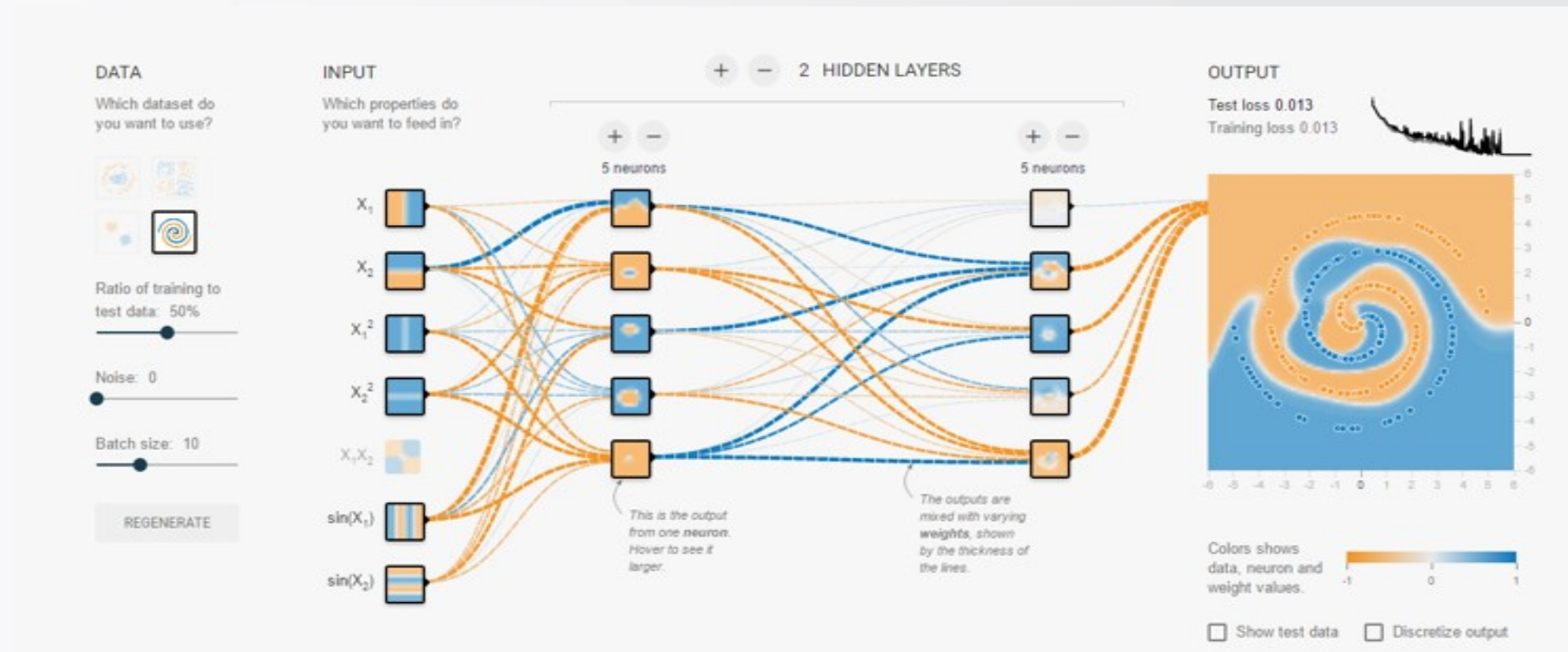


Clasificación con un MLP

Agregaremos más capas ocultas de neuronas. Cada neurona de una capa está conectada con todas las neuronas de la capa siguiente (“**full-connected**”). Hay más parámetros para ajustar por lo que el proceso de aprendizaje tarda mucho más tiempo.



En realidad, con menos capas o neuronas se pueden conseguir buenos resultados, ajustando los demás hiperparámetros que no hemos tocado aún.



Esta herramienta solo considera redes neuronales densas (full-connected). Uno de los **hiperparámetros** más difíciles de ajustar es el **número de capas** y el **número de neuronas por capa**.

Pocas neuronas en las capas ocultas produce **underfitting** (escasa cantidad de neuronas para aprender las características de un dataset complicado).

Demasiadas neuronas puede causar **overfitting** (mucha capacidad de procesamiento, cantidad limitada de información contenida en el conjunto de entrenamiento, es decir, no es suficiente para entrenar todas las neuronas de la capa oculta).

Además, se incrementaría significativamente el tiempo de entrenamiento y necesitaríamos GPUs.

Principales desafíos

Cantidad insuficiente de datos: se necesitan muchos datos para que los algoritmos funcionen correctamente, incluso para problemas muy simples.

Datos no representativos: para que el modelo generalice bien, los datos de entrenamiento deben ser representativos de los nuevos casos a los que desea generalizar.

Si la muestra es demasiado pequeña, tendrá **ruido de muestreo** (datos no representativos). Incluso muestras muy grandes pueden no ser representativas si el método de muestreo es defectuoso (**sesgo de muestreo**).

Datos de mala calidad: datos con errores, valores atípicos y ruido. Será más difícil que el sistema detecte los patrones subyacentes. Vale la pena dedicar tiempo a limpiar los datos. **"si le doy basura me devuelve basura"** (garbage in, garbage out)

¿Qué hago con los datos de mala calidad?

Valores atípicos: se descartan o se intenta corregirlos manualmente.

Características faltantes: ej. 5% de sus clientes no especificó su edad, debe decidir si desea ignorar este feature por completo, ignorar estas instancias (samples o filas del dataset), completar los valores faltantes (ej. con la edad media) o entrenar un modelo con la característica y otro sin ella.

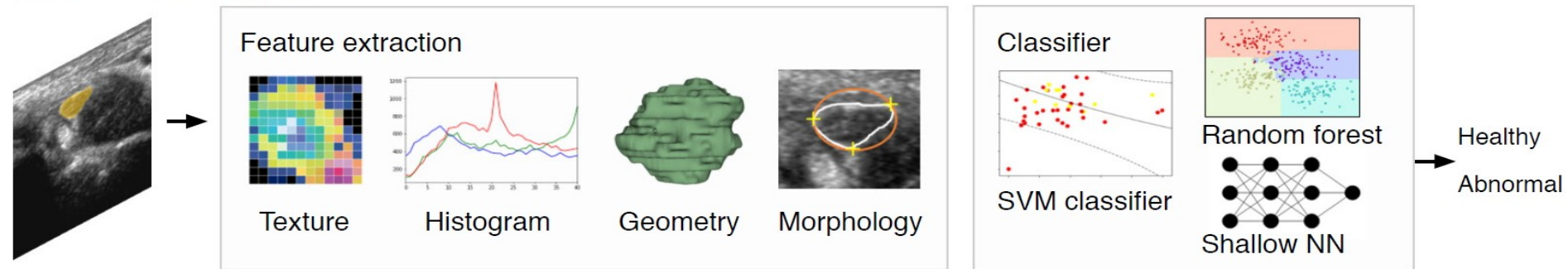
Características irrelevantes: el sistema solo podrá aprender si los datos contienen suficientes características relevantes.

Ingeniería de característica: crear un buen conjunto de features para entrenar. El proceso implica: a) **Feature selection** (seleccionar las características más útiles entre las existentes), b) **Feature extraction** (combinar características para producir una más útil), c) **crear nuevas características** (recopilar nuevos datos).

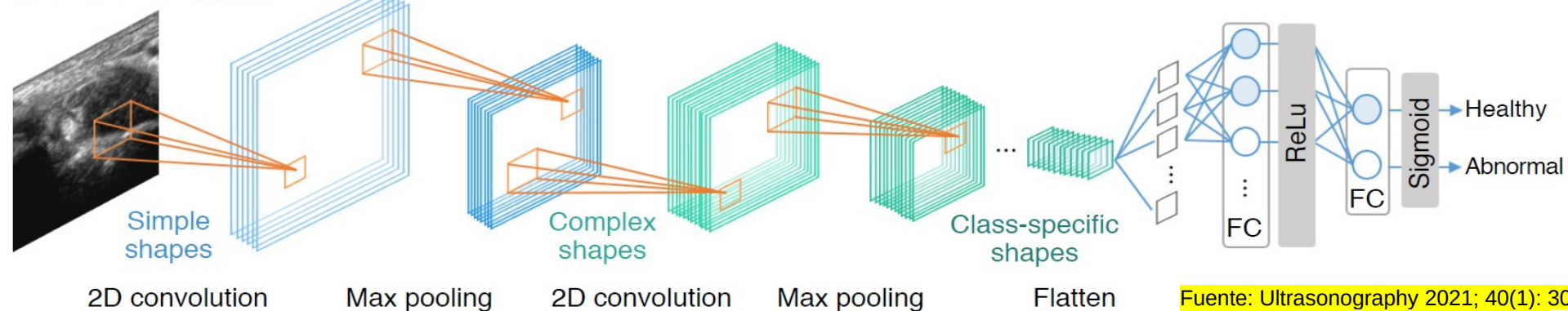
Aprendizaje Profundo

- Podemos entrenar redes neuronales con muchas más capas ocultas. Las primeras capas aprenden conceptos simples (bordes, esquinas), y las capas superiores conceptos más abstractos. Aprovecharemos el poder de cómputo paralelo de las GPU.
- Aprendizaje jerárquico: c/capa utiliza el resultado de las anteriores para construir conceptos cada vez más abstractos.
- Antes de las CNN, para cada imagen del dataset se extraían características utilizando un **feature extractor**, el cual devolvía un vector (feature vector), que alimentaba un algoritmo de clasificación (ML).
- Las CNN adoptan un enfoque diferente. En lugar de definir manualmente un conjunto de algoritmos para extraer características, estas se aprenden automáticamente durante el proceso de entrenamiento de la red.

A Machine learning (ML)



B Deep learning (DL)



Fuente: Ultrasonography 2021; 40(1): 30-44. <https://doi.org/10.14366/usg.20080>

Ejemplos de aplicaciones de DL



- ❑ **Análisis de imágenes de productos en una línea de producción para clasificarlos automáticamente**
Clasificación de imágenes [redes neuronales convolucionales, CNN]
- ❑ **Detección de tumores en imágenes médicas**
Segmentación semántica donde cada píxel de la imagen es clasificado (queremos determinar la ubicación exacta y la forma de los tumores) [CNN].
- ❑ **Clasificación automática de noticias periodísticas**
Procesamiento del lenguaje natural (NLP), clasificación de texto [redes neuronales recurrentes RNN, CNN]
- ❑ **Creación de un chatbot o un asistente personal**
Involucra muchos componentes de NLP, incluidos módulos de respuesta a preguntas y comprensión del lenguaje natural.
- ❑ **Realizar pronósticos (forecasting) de ingresos futuros de una empresa**
Regresión (predecir valores) [modelos de regresión lineal, polinómica, SVM, Random Forest o un red neuronal artificial]
- ❑ **Controlar una aplicación mediante comandos de voz**
Reconocimiento de voz, requiere procesar muestras de audio. Como son secuencias largas y complejas, normalmente son procesados utilizando RNN, CNN, etc..
- ❑ **Recomendación de un producto que pueda interesar a un cliente, basado en sus compras pasadas.**
Sistema de recomendación. Un enfoque es alimentar compras pasadas (y otra información sobre el cliente) a una ANN y pedirle que determine la próxima compra más probable.



UNIVERSIDAD
CAECE

Cámara Argentina de Comercio y Servicios